
Introduction to Machine Learning
(GAI 601)
WEEK 12
CU
NY
SPS

CUNY SPS Data Science Seminar

6:30PM Thursday, November 19th

CUNY@Amazon, Lobby 12 West 39th St. NY, NY 10018

AJ Strauman-Scott

Renaissance Philanthropy

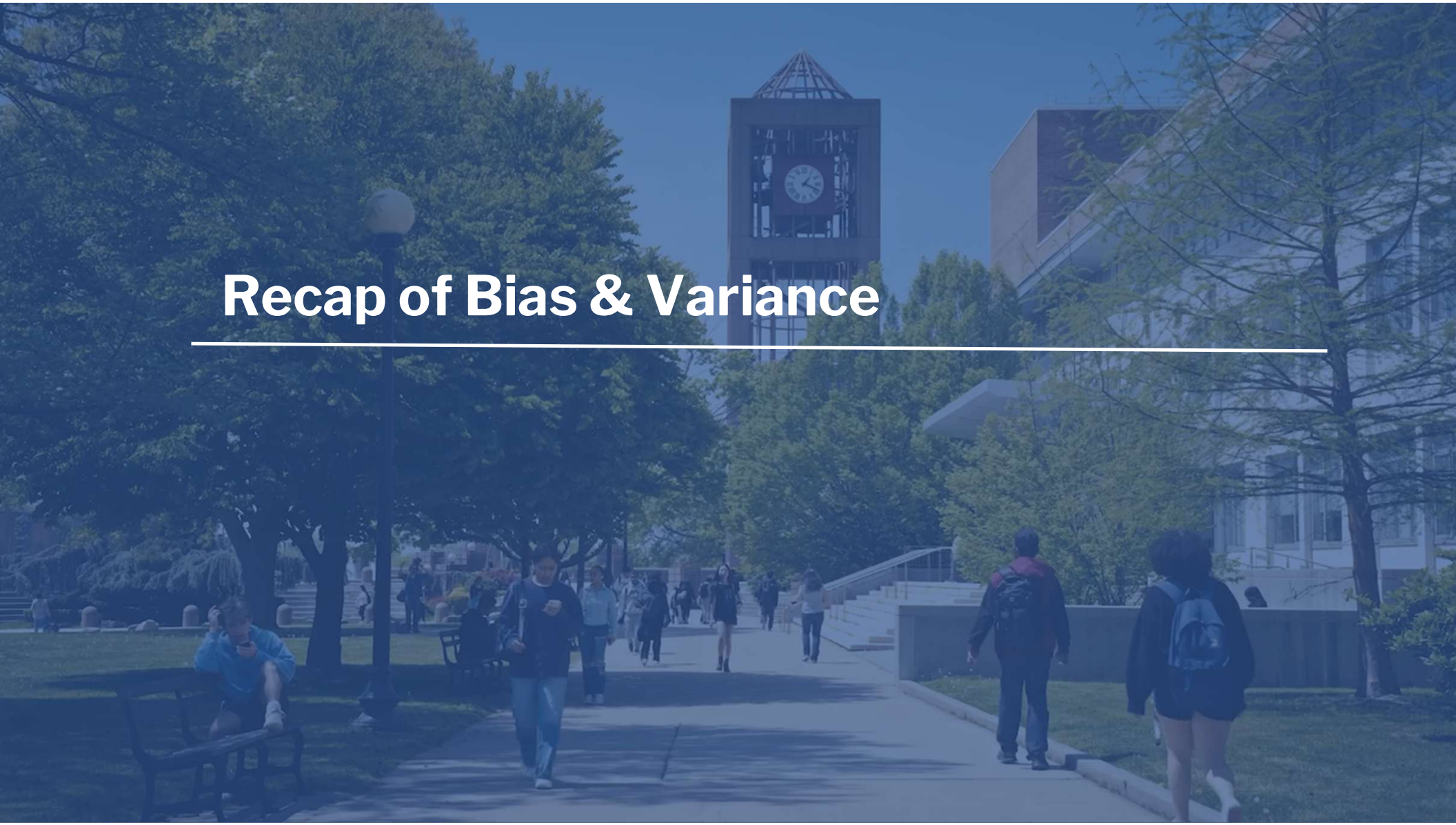
Small Voices, Big Challenges: Automatic Speech Recognition for Scaling Remote One-on-One Tutoring



CUNY THE CITY UNIVERSITY OF NEW YORK

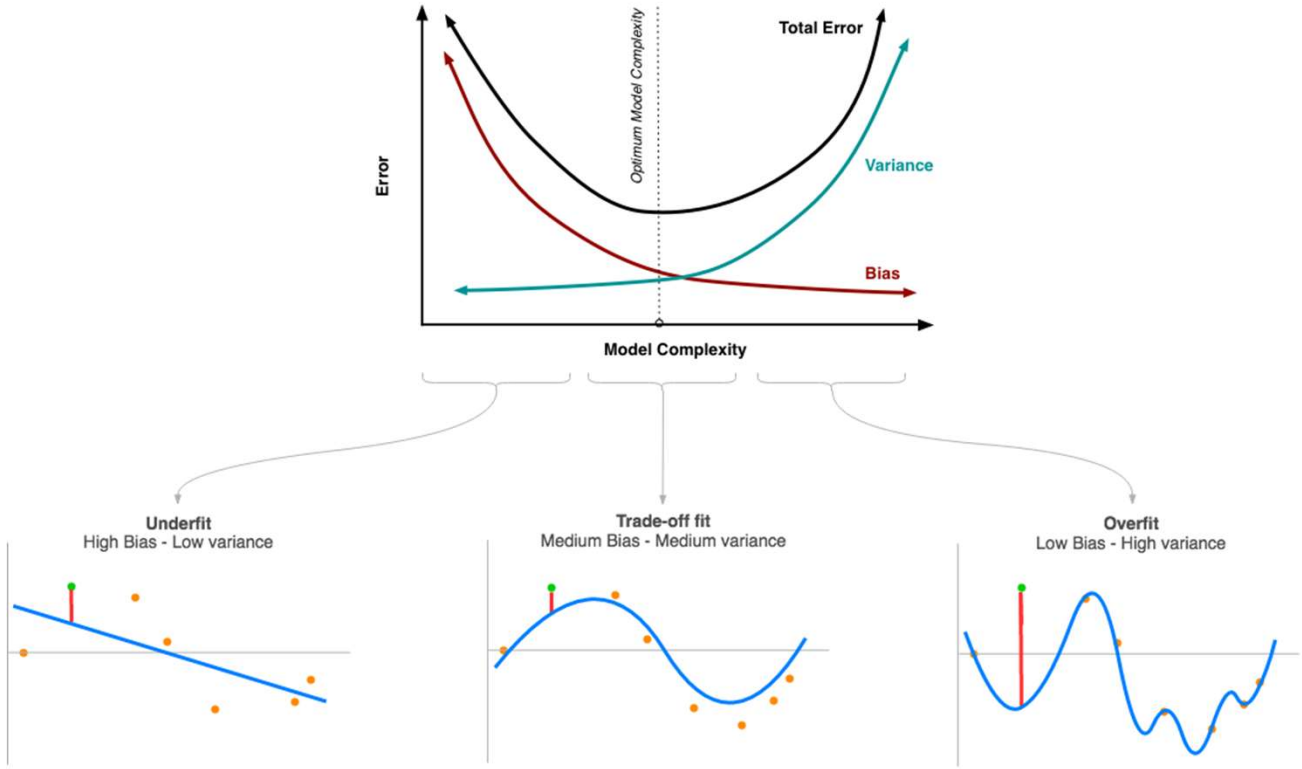
What does applied machine learning look like when the goal is to accelerate student learning? Using one-on-one tutoring as a case study, this talk follows the path from recorded conversations to transcripts, analysis, and monitoring. Along the way, we'll examine the challenges of recognizing children's speech, how transcription errors affect what we can reliably learn about students' understanding and tutors' support, and what it takes to make these pipelines reliable in real-world educational settings.

https://us02web.zoom.us/meeting/register/vNnES24tQkqUktS86HL1og



Recap of Bias & Variance

Recap: Bias-Variance Trade-off



Source http://www.ebc.cat/author/eduard-bonadagmail-com/)


Neural Networks

A Brief History of Artificial Neural Networks

Perceptron (1958)

A single neuron that can learn for classification

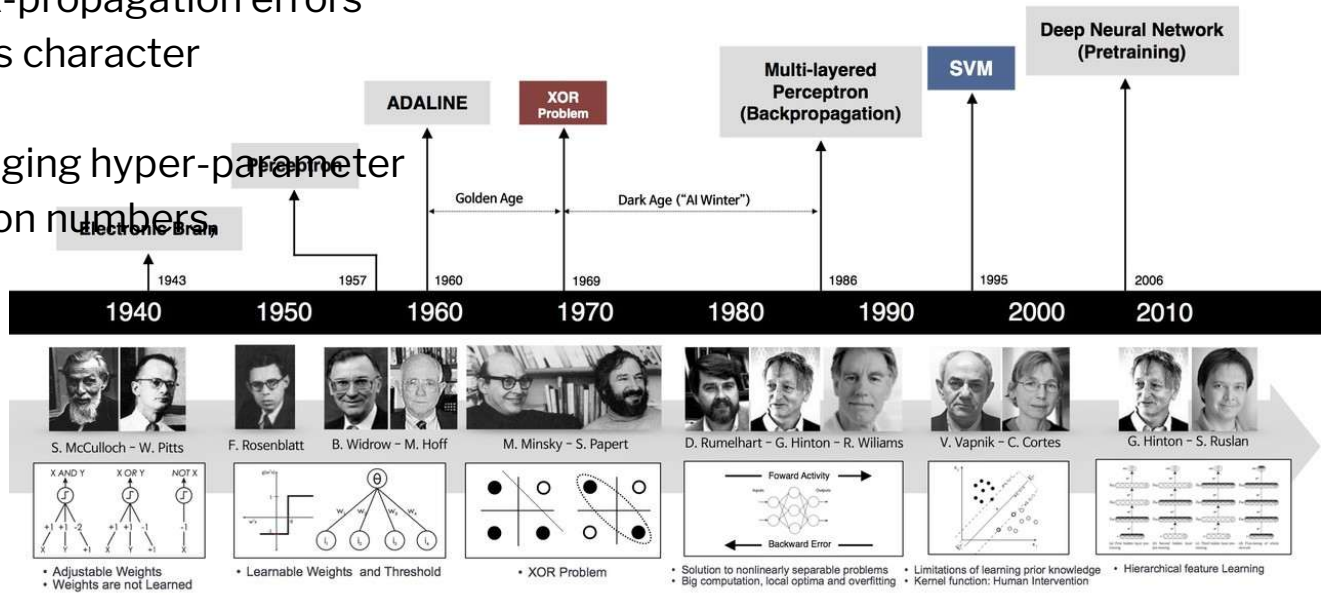
Backpropagation (1988)

Learning representations by back-propagation errors

- Successful applications such as character recognition
- Limitations: overfitting, challenging hyper-parameter tuning such as number of neuron numbers, number of layers, and so on.

Deep Learning (2006)

- Autoencoder
- Convolutional Neural Networks
- Recurrent Neural Networks



Source: Nvidia

Structure of Neurons in Human Brain

Dendrite

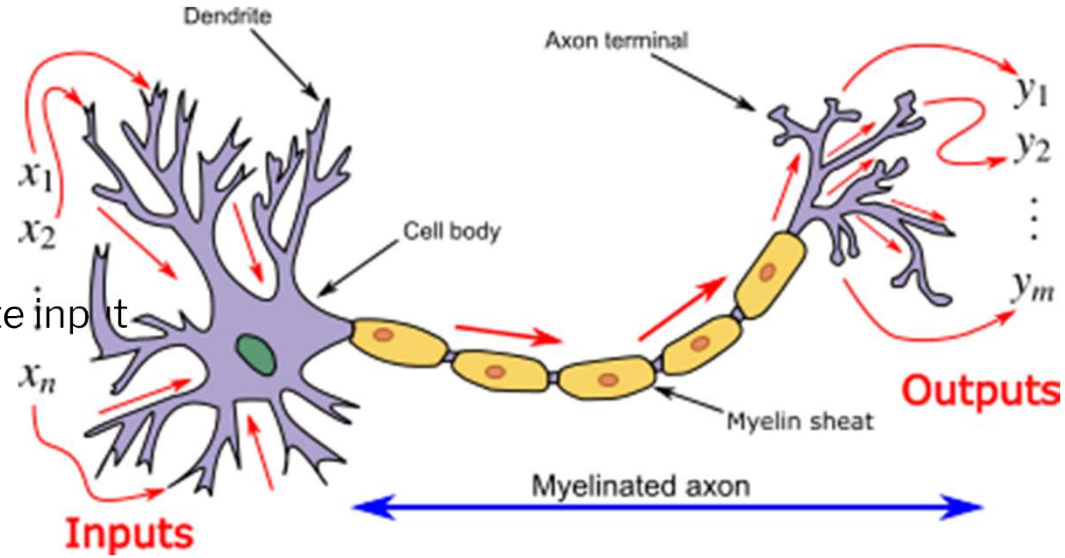
Receives signals from other neurons

Cell body

Sums all the incoming signals to generate input

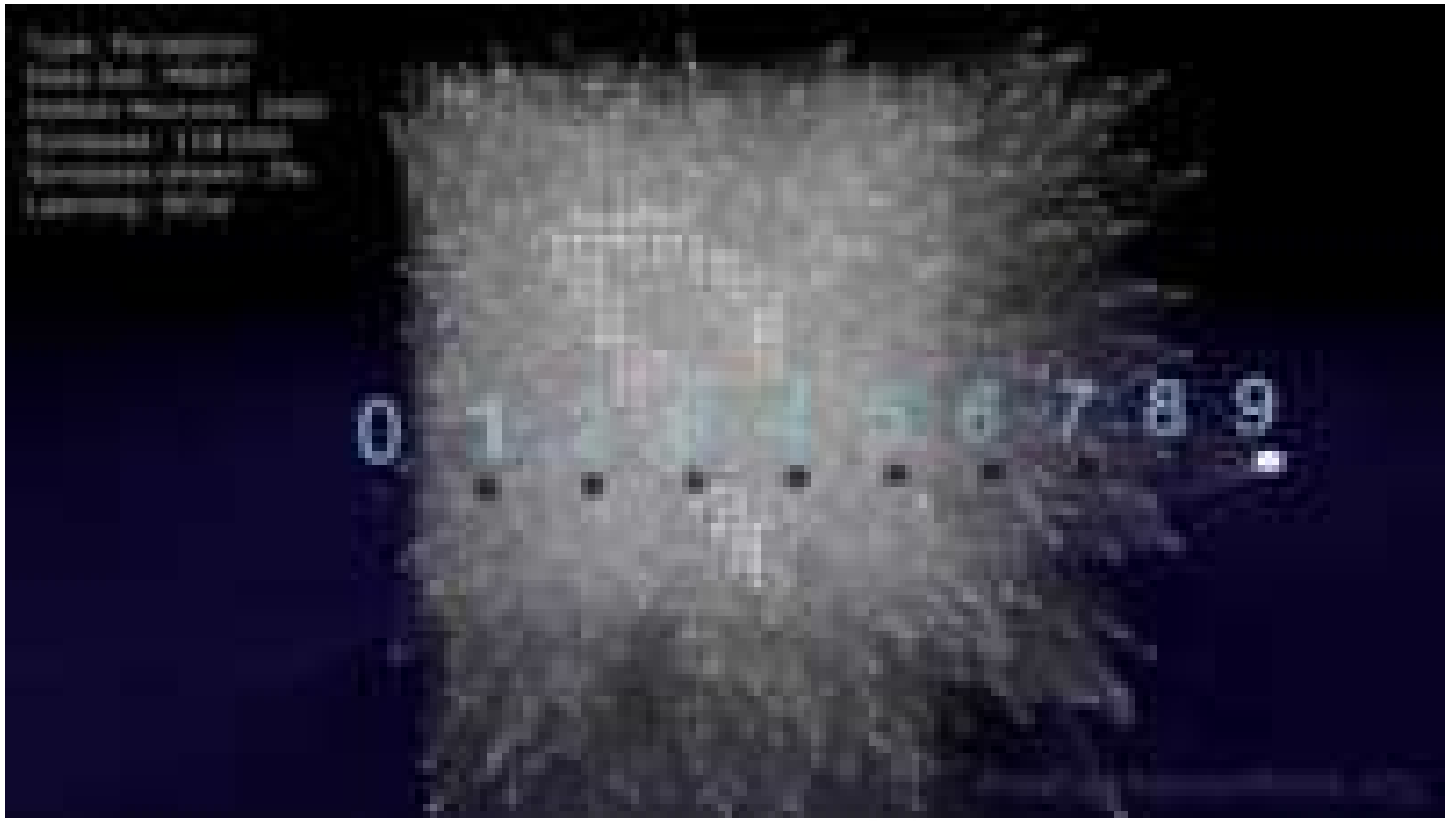
Axon

When the sum reaches a threshold value, neuron fires and the signal travels down the axon to the other neurons



Source: Nvidia

Neural networks Visualization



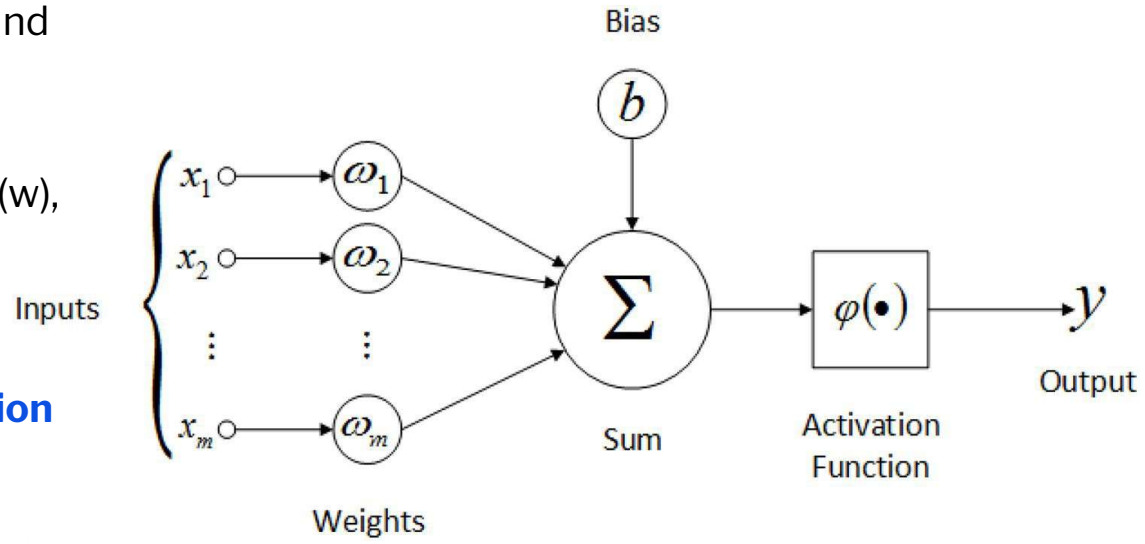
https://youtu.be/3JQ3hYko51Y?t=10

Basic Computational Unit

Artificial neuron, often called a node or unit, receives input from some other neurons, or from an external source and computes an output.

Each input has an associated **weight** (w), which is assigned on the basis of its relative importance to other inputs.

The node applies an **activation function** to the weighted sum of its inputs.



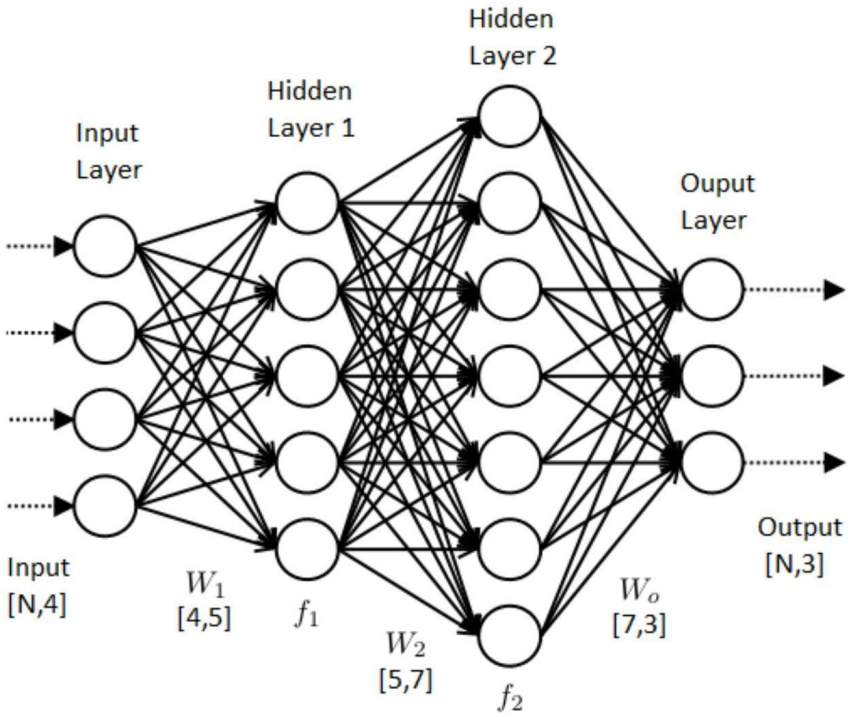
Source: Source: https://dzone.com/articles/the-artificial-neural-networks-handbook-part-4

Feedforward Neural Network

A **feedforward neural network** is an artificial neural network where connections between the units do not form a cycle.

In this network, the information moves in only one direction, forward, from the **input nodes**, through the **hidden nodes** (if any) and to the **output nodes**.

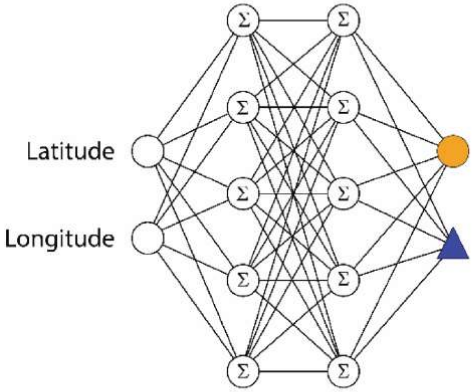
No cycles or loops in the network.



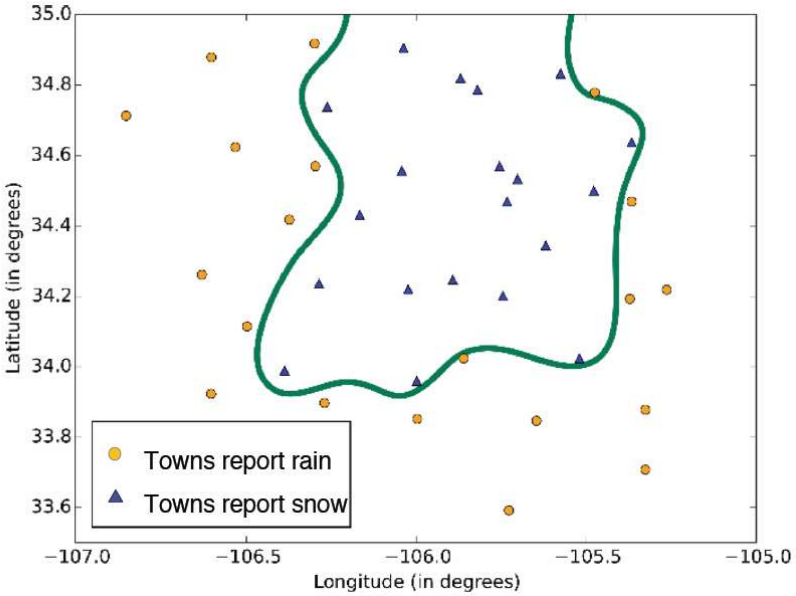
Source: https://dzone.com/articles/the-artificial-neural-networks-handbook-part-1-1

Applications

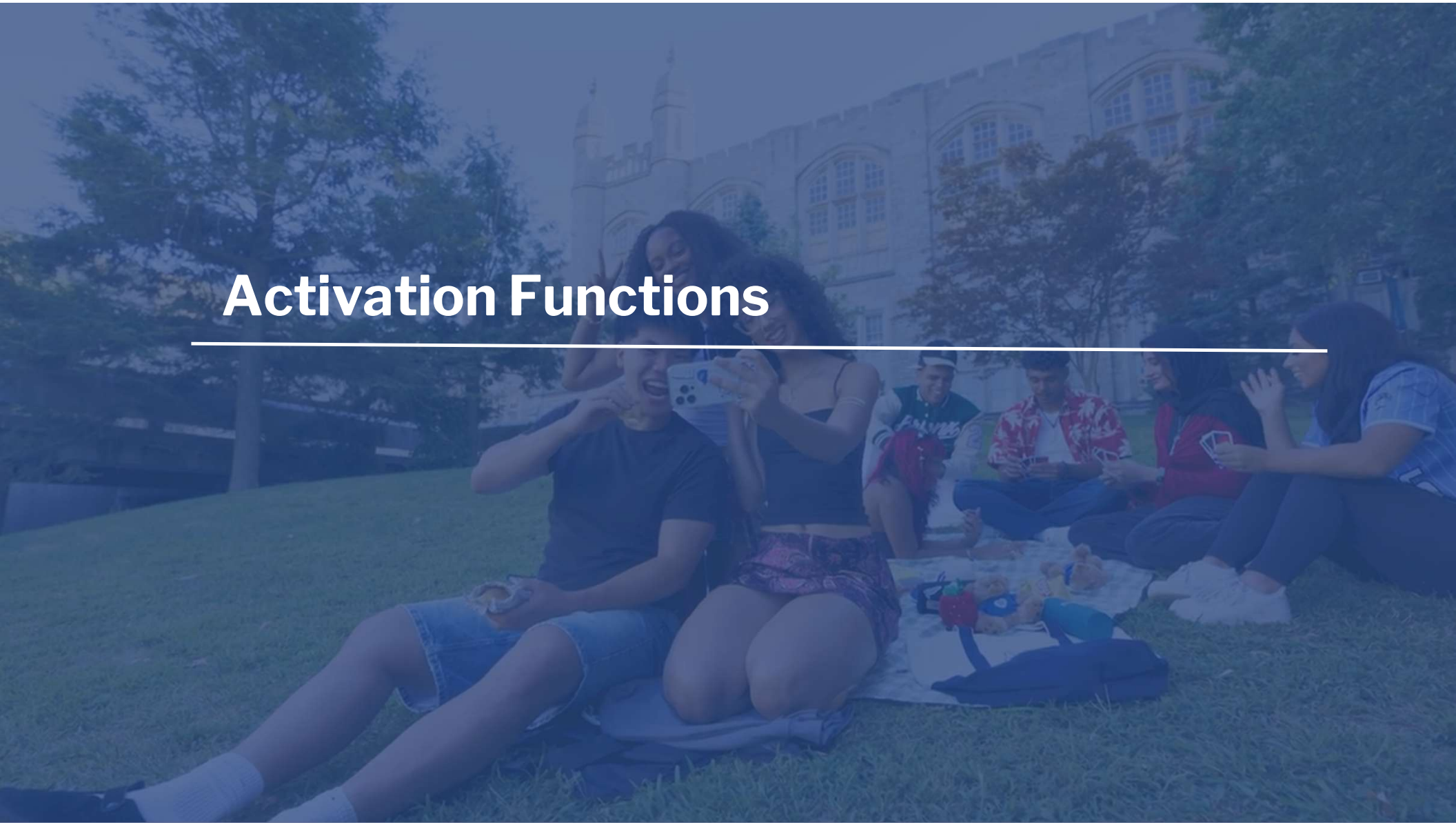
- For modeling highly nonlinear systems
- When data is available incrementally and you wish to constantly update the model
- When there could be unexpected changes in your input data
- When model interpretability is not a key concern



Neural network for classification problem



Source: Source: https://www.researchgate.net/figure/Some-applications-of-AI-neural-network-accelerators_fig1_343711174



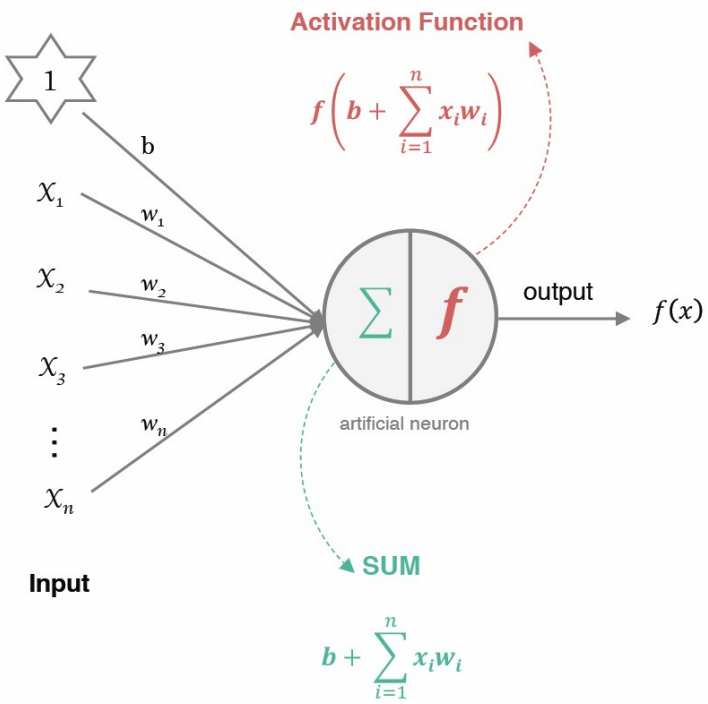
Activation Functions

How does artificial neuron work ?

- Artificial neurons are most basic units of information processing in an artificial neural network.
- Each neuron takes information from a set of neurons, processes it, and gives the result to another neuron for further processing.
- All the inputs x are multiplied with their weights w, and added together

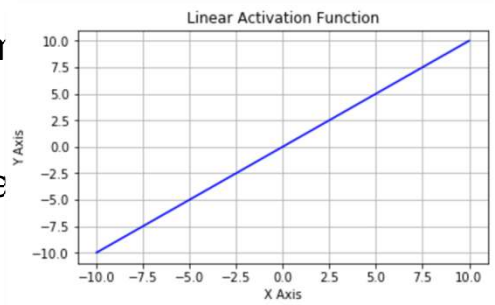
$$weighted\ sum = b*1 + x_1*w_1 + \ldots + x_n*w_n = b + \sum_{i=1}^{n} x_i w_i$$

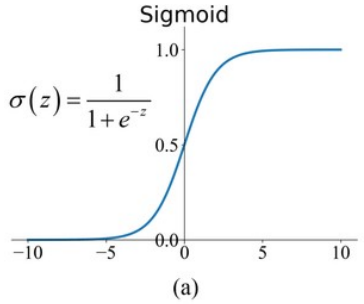
- The weighted sum is passed to activation function $f(x) = f\left(b + \sum_{i=1}^{n} x_i w_i\right)$

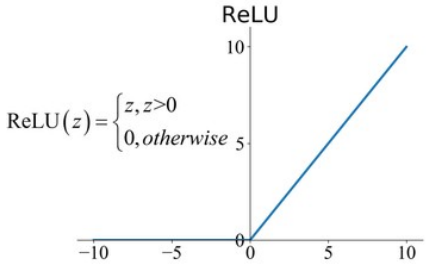


Activation Functions

- Activation functions provide non-linear transform ("reshape" data)
- Biologically inspired, it is usually an abstraction re the rate of action potential firing in the cell
- Sigmoid is typically used for classification e.g. to predict the probability, as an output.
- Rectified Linear Unit (ReLU) is the 'go-to' choice for most purposes.
- There are a lot of choices of activation functions – these are only two of many – but they must always be non-linear.

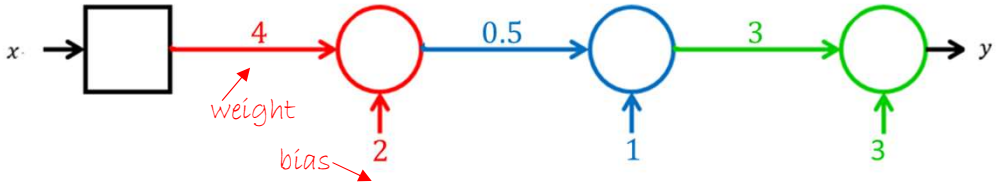




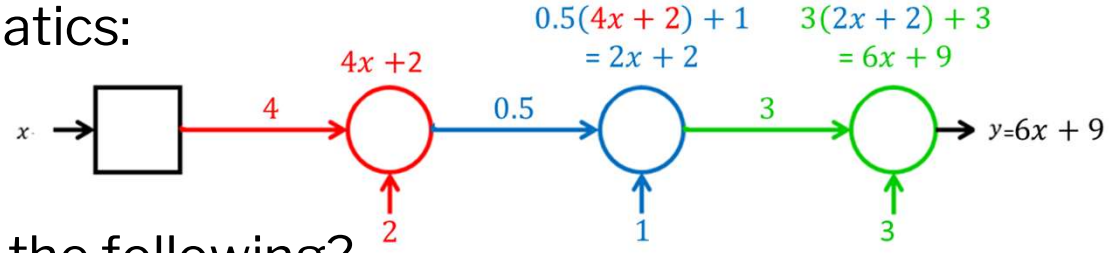


Do I really need activation functions?

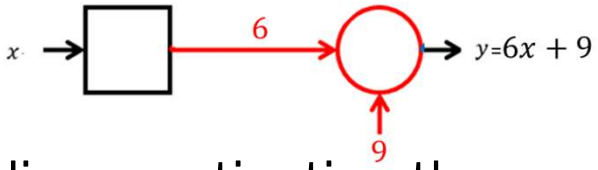
Let's consider a simple neural network:



Let's expand the mathematics:



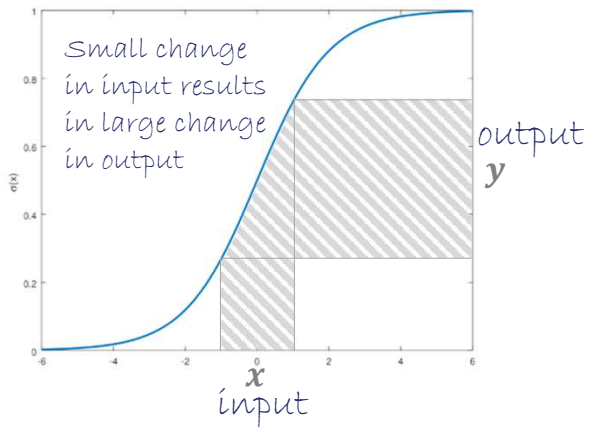
But isn't that the same as the following?

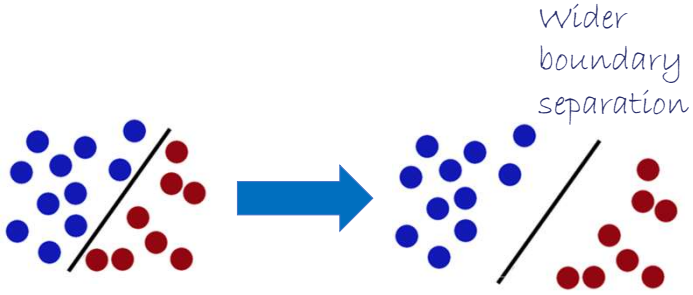


Takeaway: Without a non-linear activation the neural network will collapse

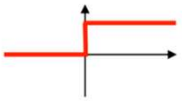
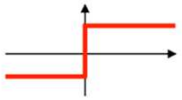
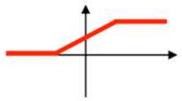
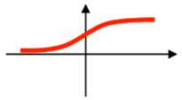
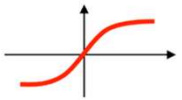
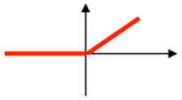
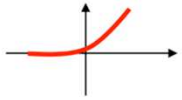
Sigmoid Activation Function

- Useful for output layer for classification problems
- A small change in input causes large change in output, at decision boundary
- Separates classes away from decision boundary
- Softmax performs a similar function for multi-class classification (3 or more classes)



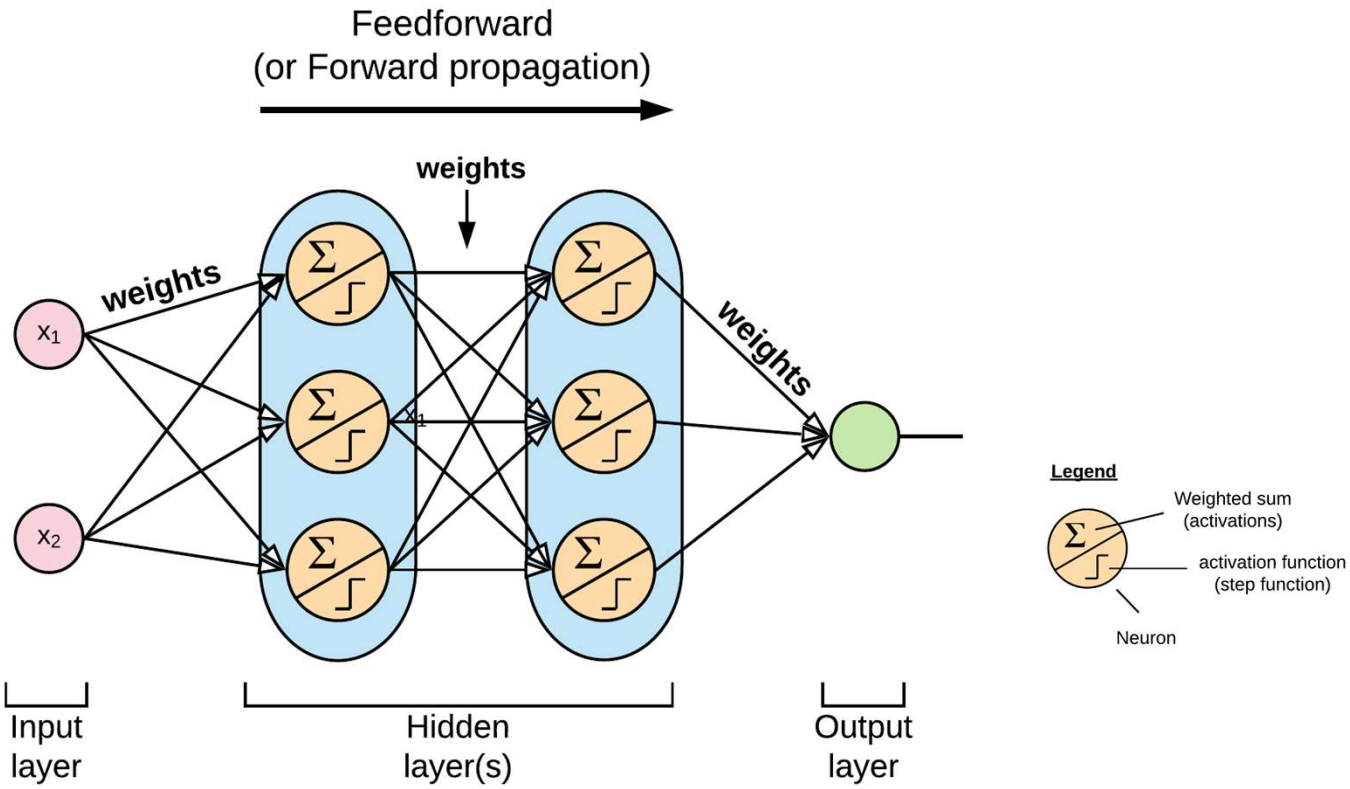


More Activation Functions to choose from

Activation function	Equation	Example	1D Graph
Unit step (Heaviside)	$\phi(z) = \begin{cases} 0, & z < 0, \\ 0.5, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Sign (Signum)	$\phi(z) = \begin{cases} -1, & z < 0, \\ 0, & z = 0, \\ 1, & z > 0, \end{cases}$	Perceptron variant	
Piece-wise linear	$\phi(z) = \begin{cases} 1, & z \geq \frac{1}{2}, \\ z + \frac{1}{2}, & -\frac{1}{2} < z < \frac{1}{2}, \\ 0, & z \leq -\frac{1}{2}, \end{cases}$	Support vector machine	
Logistic (sigmoid)	$\phi(z) = \frac{1}{1 + e^{-z}}$	Logistic regression, Multi-layer NN	
Hyperbolic tangent	$\phi(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$	Multi-layer Neural Networks	
Rectifier, ReLU (Rectified Linear Unit)	$\phi(z) = max(0, z)$	Multi-layer Neural Networks	
Rectifier, softplus	$\phi(z) = \ln(1 + e^{z})$	Multi-layer Neural Networks	

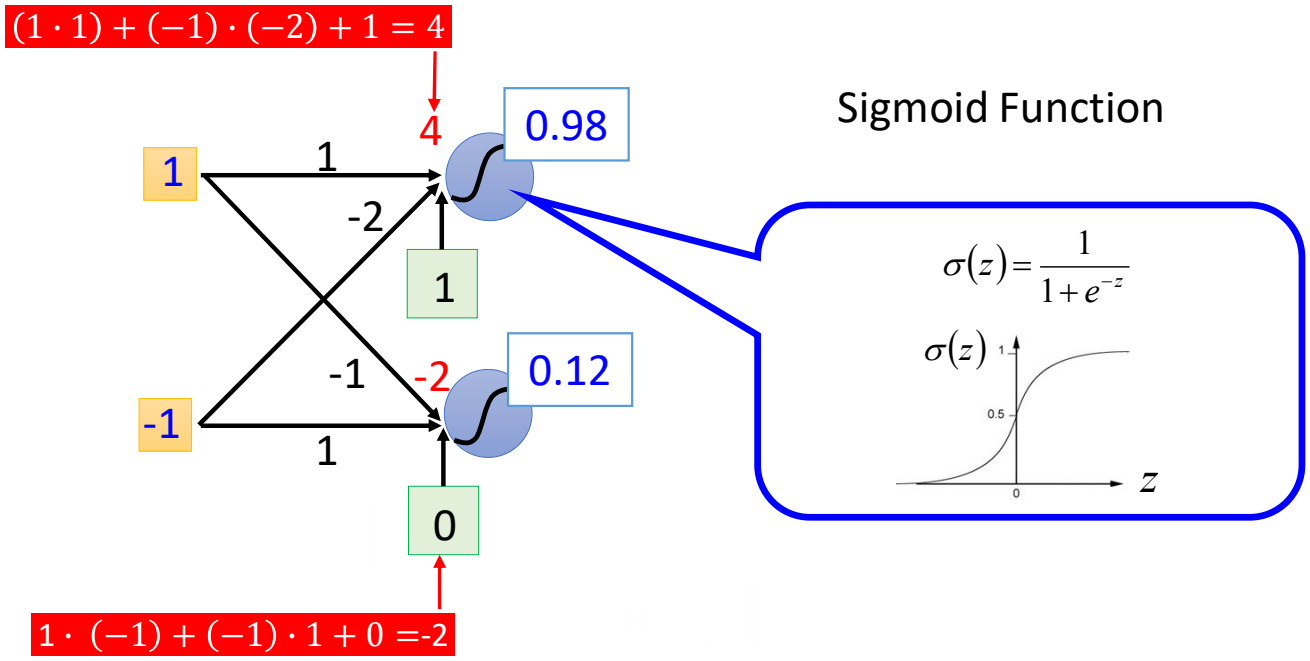

Feed Forward Networks

Feed Forward



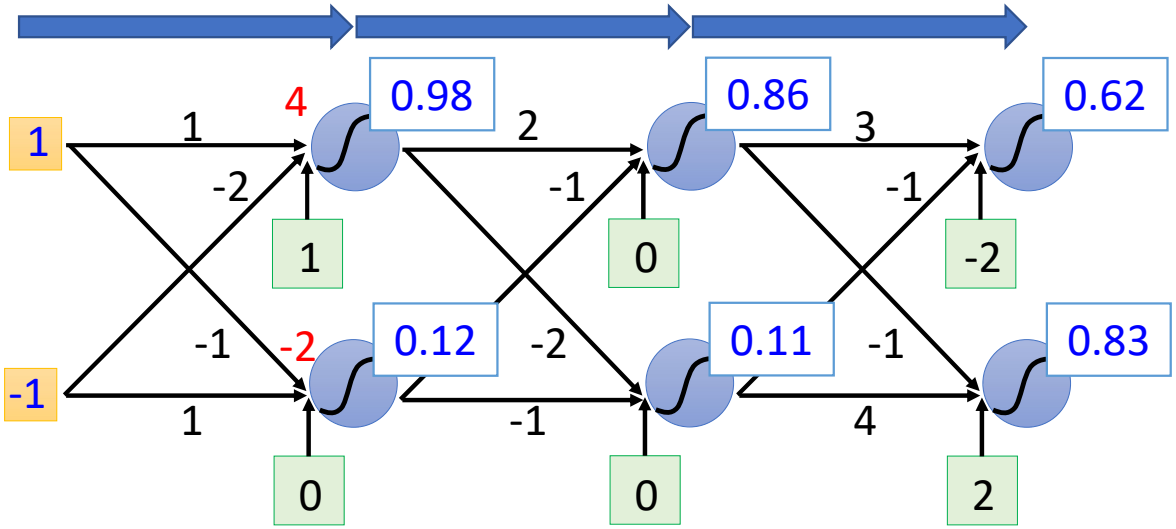
Feedforward

A simple network, toy example



Source: Hung-yi Lee – Deep Learning Tutorial

Feedforward

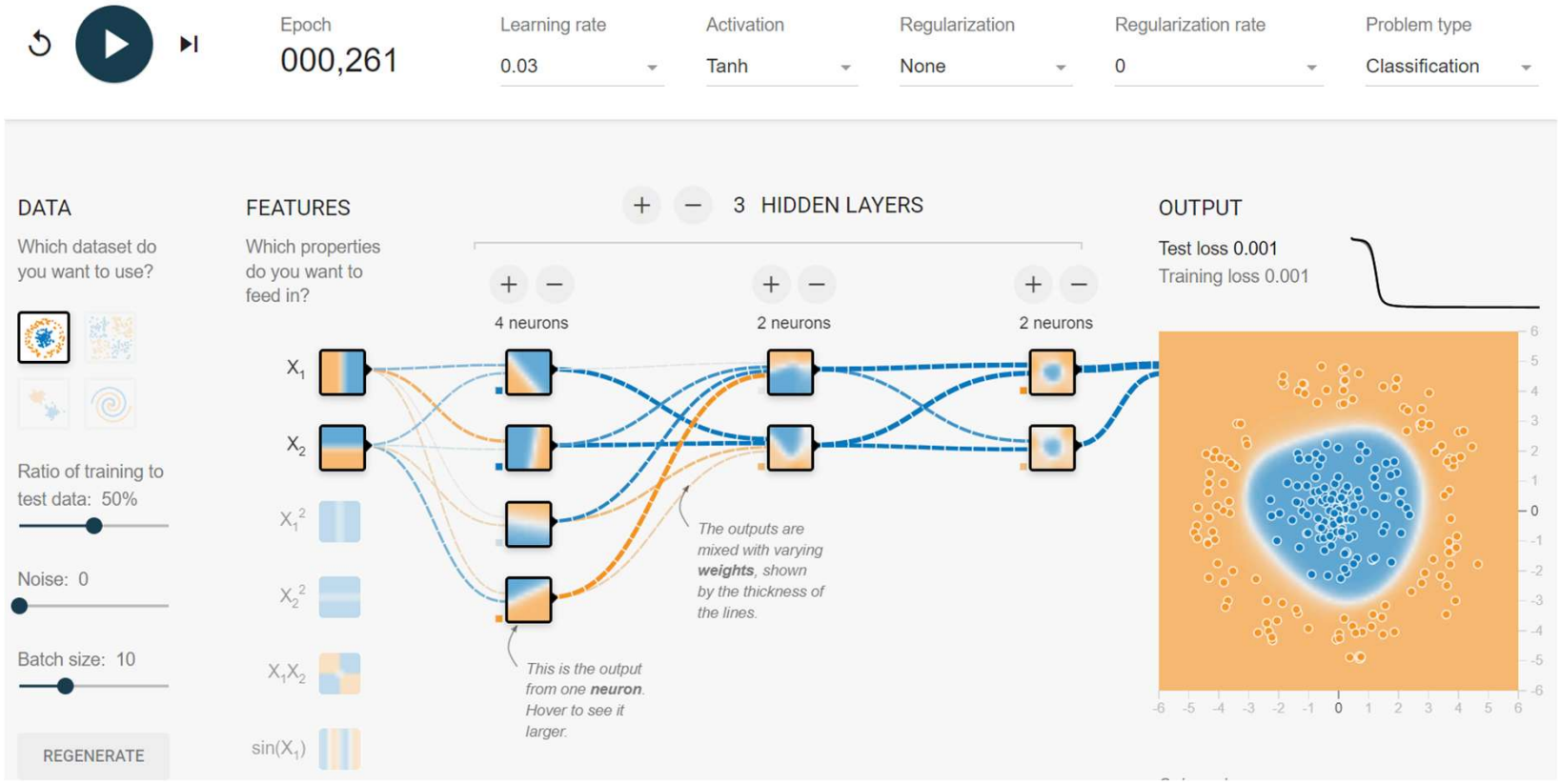


$$f: R^2 \rightarrow R^2 \qquad f\left(\begin{bmatrix}1\\-1\end{bmatrix}\right) = \begin{bmatrix}0.62\\0.83\end{bmatrix}$$

Source: Hung-yi Lee – Deep Learning Tutorial

Neural network demo

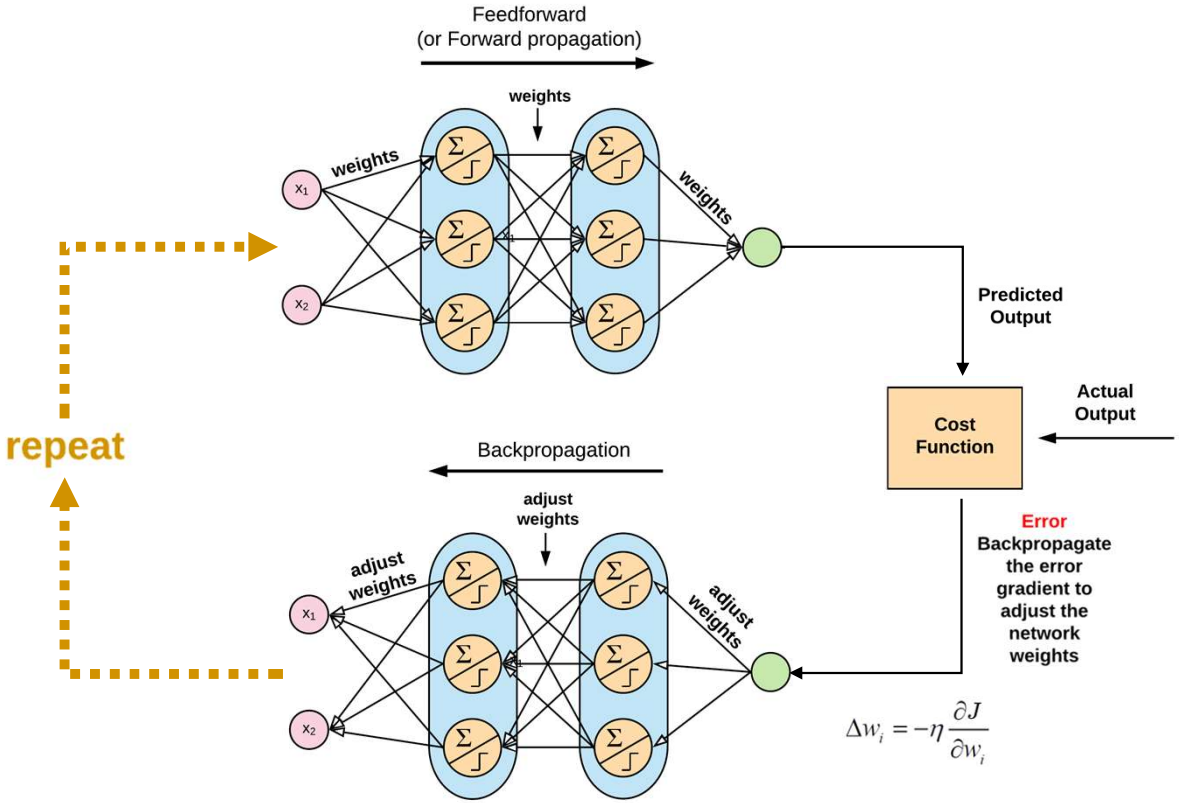
https://playground.tensorflow.org/





Backpropagation

Backpropagation

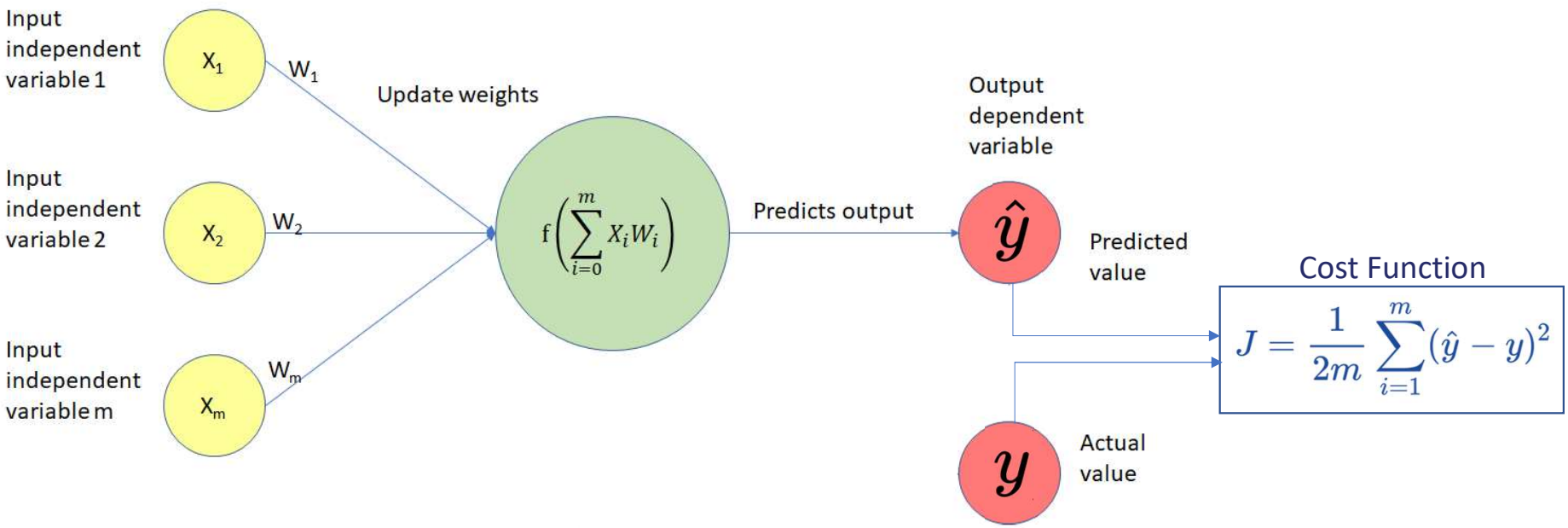


Source: https://ekababisong.org/gcp-ml-seminar/deep-learning/

Backpropagation

- Backpropagation is a type of **gradient descent** (terms used interchangeably)
- Gradient descent refers to the calculation of a gradient on each weight in the neural network for each training element.
- Because the neural network will not output the expected value for a training element, the gradient of each weight will give you an indication about how to modify each weight to achieve the expected output.
- If the neural network did output exactly what was expected, the gradient for each weight would be 0, indicating that no change to the weight is necessary.
- How it works:
  - Output of NN is evaluated against desired output
  - If results are not satisfactory, connection (weights) between layers are modified and process is repeated again and again until error is small enough.

Cost Function (J)



Cost Function

- Cost functions determine how well a model performs for a given dataset
- Cost Functions measure just how wrong the model is in finding a relation between the input and output.
- Comparing other functions:
  - Accuracy functions tell you how well your model does, not how to improve it
  - Error functions measure the difference between the target and the actual values e.g. $(\hat{y} - y)$ $(\hat{y} - y)^2$
  - Loss functions quantify the cost for a single training example e.g.
  - Cost functions quantify the average across the entire dataset e.g. $J = \frac{1}{2m} \sum_{i=1}^{m} (\hat{y} - y)^2$

  - Loss functions quantify the impact of the error i.e. error is objective while loss is subjective e.g. we might adopt a non-symmetric loss function if we may be more negatively affected by an error in a particular direction (e.g., false positive vs. false negative)

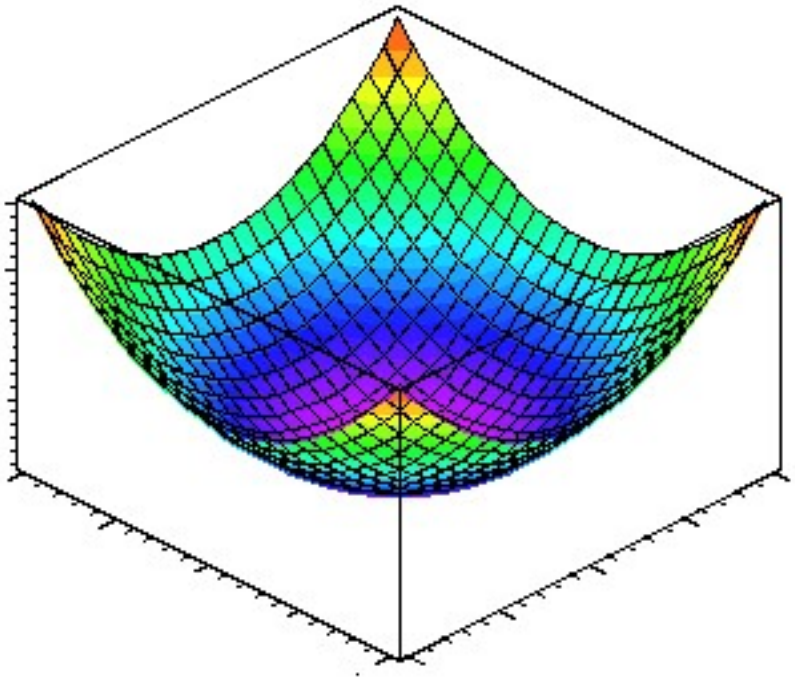
Cost Function

MSE Cost function will always be parabolic (b
So it has only one global minimum

$$J = \frac{1}{2m}\sum_{i=1}^{m}(\hat{y} - y)^2$$

- SSE – Similar to MSE but not an average

$$J = \frac{1}{2m}\sum_{i=1}^{m}(\hat{y} - y)^2$$



Note: The 2 we added in the denominator makes the mathematics cleaner.

Gradient Descent Rule

Gradient descent rule:

$$w_i \leftarrow w_i + \Delta w_i$$

Where:

$$\Delta w_i = -\eta \frac{\partial J}{\partial w_i}$$

η (eta) is a positive constant called the **learning rate**, and determines step size of gradient descent search

Cost Function

$$SSE = \frac{1}{2}\sum_{j=1}^{n}(y_j - \hat{y}_j)^2$$

$$SSE = \frac{1}{2}\sum_{i=1}^{n}\left(y_j - \left(\sum_{i=0}^{m}\phi(w_i^T \times x_{j,i})\right)\right)^2$$

$$\frac{\partial\, SSE}{\partial\, w_i} = \sum_{j=1}^{n}\left(\underbrace{(y_j - \hat{y}_j)}_{\substack{\text{error of the}\\ \text{output of the}\\ \text{weighted sum}}} \times \underbrace{-x_{j,i}}_{\substack{\text{rate of change of}\\ \text{weighted sum}\\ \text{with respect to}\\ \text{change in } w_i}}\right)$$

Derivative of the Sigmoid Function

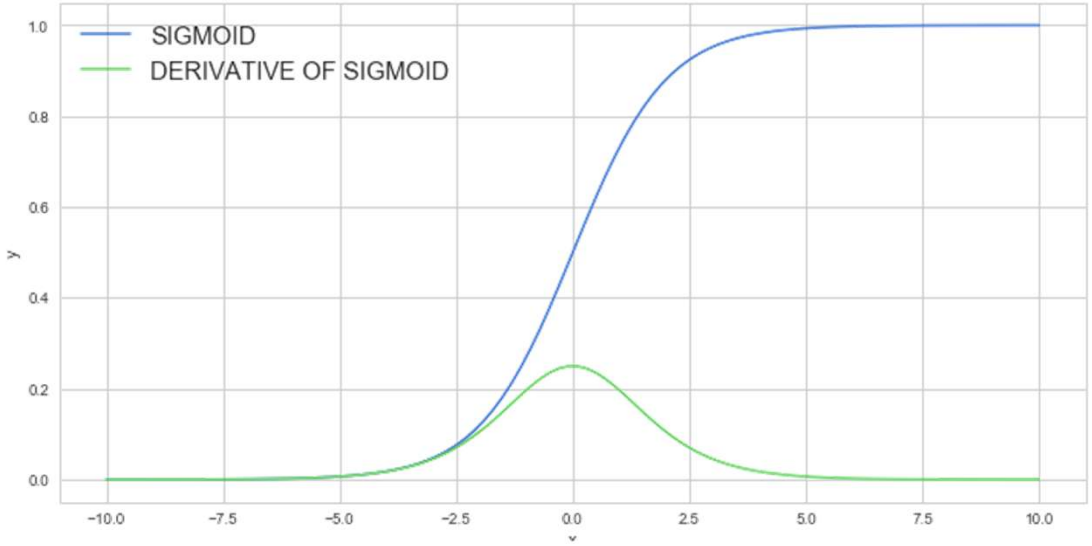
$$y = \frac{1}{1+e^{-x}}$$

$$\frac{dy}{dx} = -\frac{1}{(1+e^{-x})^2}(-e^{-x})$$

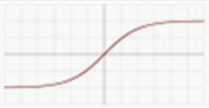
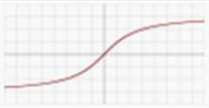
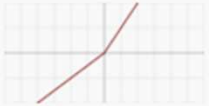
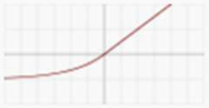
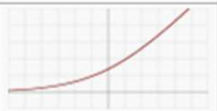
$$= \frac{e^{-x}}{(1+e^{-x})^2}$$

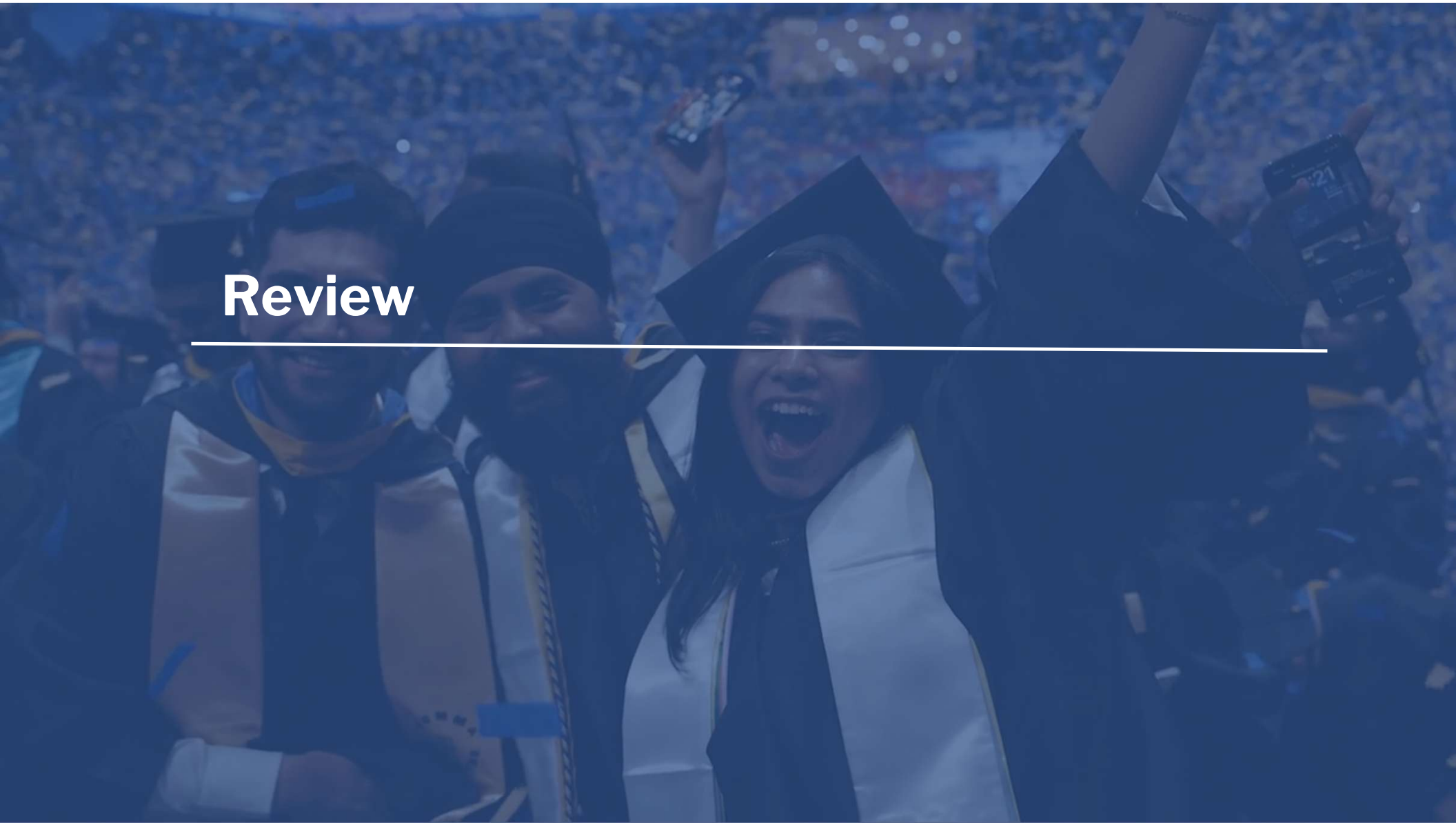
$$= \frac{1}{1+e^{-x}}\left(1-\frac{1}{1+e^{-x}}\right)$$

$$= y(1-y)$$



Derivatives of other Activation Functions

Name	Plot	Equation	Derivative
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$
TanH		$f(x) = \tanh(x) = \frac{2}{1+e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
ArcTan		$f(x) = \tan^{-1}(x)$	$f'(x) = \frac{1}{x^2+1}$
Rectified Linear Unit (ReLU)		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Parameteric Rectified Linear Unit (PReLU) [2]		$f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Exponential Linear Unit (ELU) [3]		$f(x) = \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} f(x) + \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
SoftPlus		$f(x) = \log_e(1 + e^x)$	$f'(x) = \frac{1}{1+e^{-x}}$



Review

This week we covered

Lesson Objectives/Topics

- **Recap of Bias & variance**
  - Explain how bias and variance contribute to total prediction error
  - Explain the trade-off between bias and variance in predictive modeling
  - Adjust model complexity using regularization techniques
  - Interpret the bias-variance tradeoff in real-world business scenarios
- **Deep Learning**
  - Explain basic neural network architectures for classification or regression
  - Explain the role of hidden layers and activation functions in model learning
  - Compare foundation models to traditional ML approaches for scalability
  - Identify scenarios where neural networks provide business value over simpler models


CUNY | School of
Professional Studies